Rod Ciombor
4/13/26
CIS-2532-NET01
Dr. Sheikh Shamsuddin
Week 9 Lab

Functions library contents
Here is a list of the contents in **week9hw.py**

In [ ]:
```python
# Author:       Rod Ciombor
# Date:         04/13/2026
# Instructor:   Dr. Sheikh Shamsuddin
# Class:        CIS-2532-NET01

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
import os

SCRIPT_DIR = os.path.dirname(os.path.abspath(__file__))
TITANIC = os.path.join(SCRIPT_DIR, 'titanic.csv')
WORKERS_TIPS = os.path.join(SCRIPT_DIR, 'workerstips.csv')
FLIGHTS_DATA = os.path.join(SCRIPT_DIR, 'flightsData.csv')

def questionA():
    df = pd.read_csv(WORKERS_TIPS)
    sns.scatterplot(data=df, x="total_bill", y="tip")
    plt.show()

def questionB(size_var):
    df = pd.read_csv(WORKERS_TIPS)
    sns.scatterplot(
        data=df,
        x="total_bill",
        y="tip",
        hue="smoker",
        style="smoker",
        size=size_var,
        sizes=(10, 300)
    )
    plt.show()

def questionC():
    df = pd.read_csv(WORKERS_TIPS)
    sns.barplot(
        data=df,
        x="day",
        y="tip",
        estimator="mean",
        order=["Thur", "Fri", "Sat", "Sun"],
```

```
        hue="day",
        palette={
            "Thur": "blue",
            "Fri": "orange",
            "Sat": "green",
            "Sun": "red"
        },
        legend=False
    )
    plt.show()

def questionD():
    df = pd.read_csv(WORKERS_TIPS)

    df["time"] = pd.Categorical(
        df["time"],
        categories=df["time"].unique()[::-1],
        ordered=True
    )

    sns.boxplot(
        data=df,
        x="day",
        y="tip",
        hue="time",
        order=["Thur", "Fri", "Sat", "Sun"],
        palette={
            "Lunch": "blue",
            "Dinner": "orange"
        },
        flierprops=dict(
            marker='D',
            markerfacecolor='black',
            markeredgecolor='black',
            markersize=6
        )
    )
    plt.show()

def questionE():
    df = pd.read_csv(FLIGHTS_DATA)

    sns.lineplot(
        data=df,
        x="year",
        y="passengers"
    )
    sns.lineplot(
        data=df,
        x="year",
        y="passengers",
        estimator="sum"
    )
    plt.show()
```

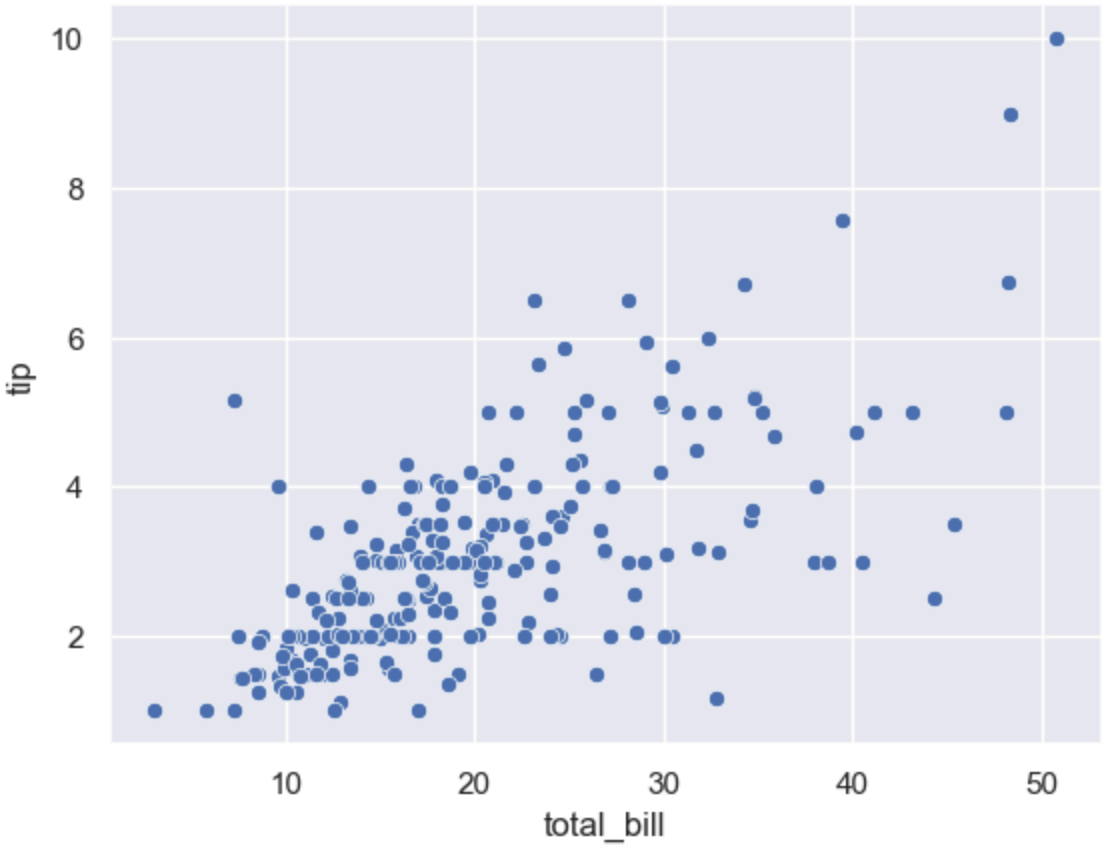
```
def questionF():

    sns.set_theme(style="darkgrid")

    hue_order = ["man", "woman", "child"]

    palette = {
        "man": "blue",
        "woman": "orange",
        "child": "green"
    }

    df = pd.read_csv(TITANIC)

    # Create category column
    df["who"] = df.apply(getAgeCategory, axis=1)

    df["Pclass"] = df["Pclass"].map({
        1: "First",
        2: "Second",
        3: "Third"
    })

    # Create side-by-side plots
    fig, axes = plt.subplots(1, 2, figsize=(12, 5), sharey=True)

    # Left: Survived = 0
    sns.countplot(
        data=df[df["Survived"] == 0],
        x="Pclass",
        hue="who",
        order=["First", "Second", "Third"],
        hue_order=hue_order,
        palette=palette,
        ax=axes[0]
    )
    axes[0].set_title("survived = 0")
    axes[0].set_xlabel("class")
    axes[0].set_ylabel("count")

    # Right: Survived = 1
    sns.countplot(
        data=df[df["Survived"] == 1],
        x="Pclass",
        hue="who",
        order=["First", "Second", "Third"],
        hue_order=hue_order,
        palette=palette,
        ax=axes[1]
    )
    axes[1].set_title("survived = 1")
    axes[1].set_xlabel("class")
    axes[1].set_ylabel("")

    for ax in axes:
        leg = ax.legend_
```

```
        if leg:
            leg.get_frame().set_facecolor("white")
            leg.get_frame().set_alpha(1)

    plt.tight_layout()
    plt.show()


def getAgeCategory(row):
    if row["Age"] < 18:
        return "child"
    elif row["Sex"] == "male":
        return "man"
    else:
        return "woman"
```

Question A Output

Here is the output for question A

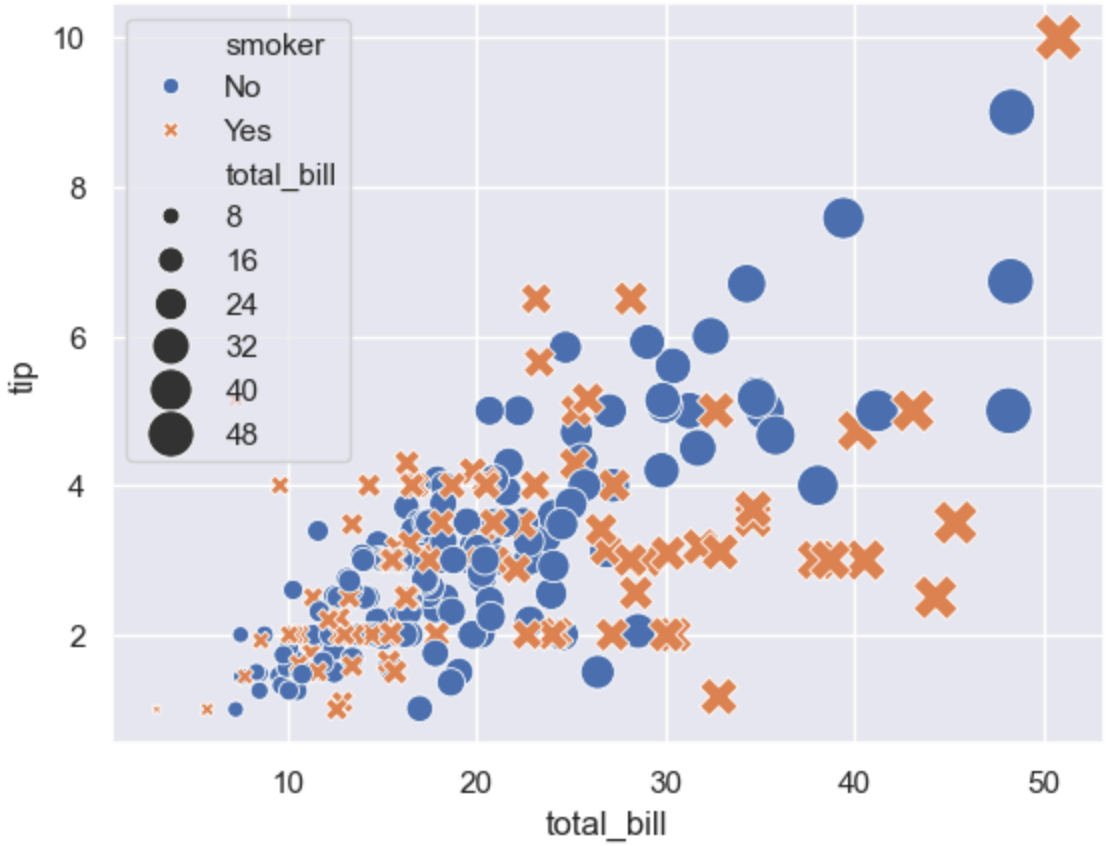
In [2]:
```
import week9hw as lib
lib.questionA()
```



Question B Output - version 1

Here is the output for question B. In this version, the size of the marks are determined by the Total Bill.

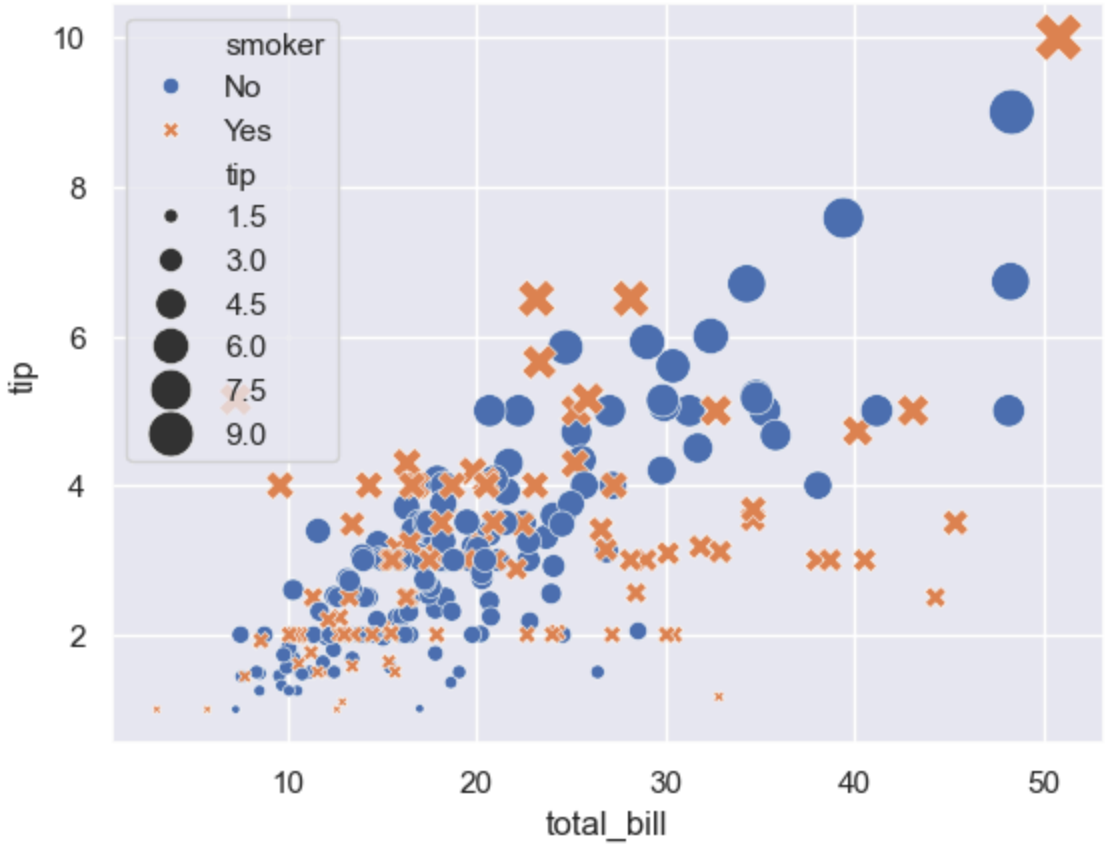
In [4]:
```
import week9hw as lib
lib.questionB("total_bill")
```



Question B Output - version 2

Here is the output for question B. In this version, the size of the marks are determined by the Tip.

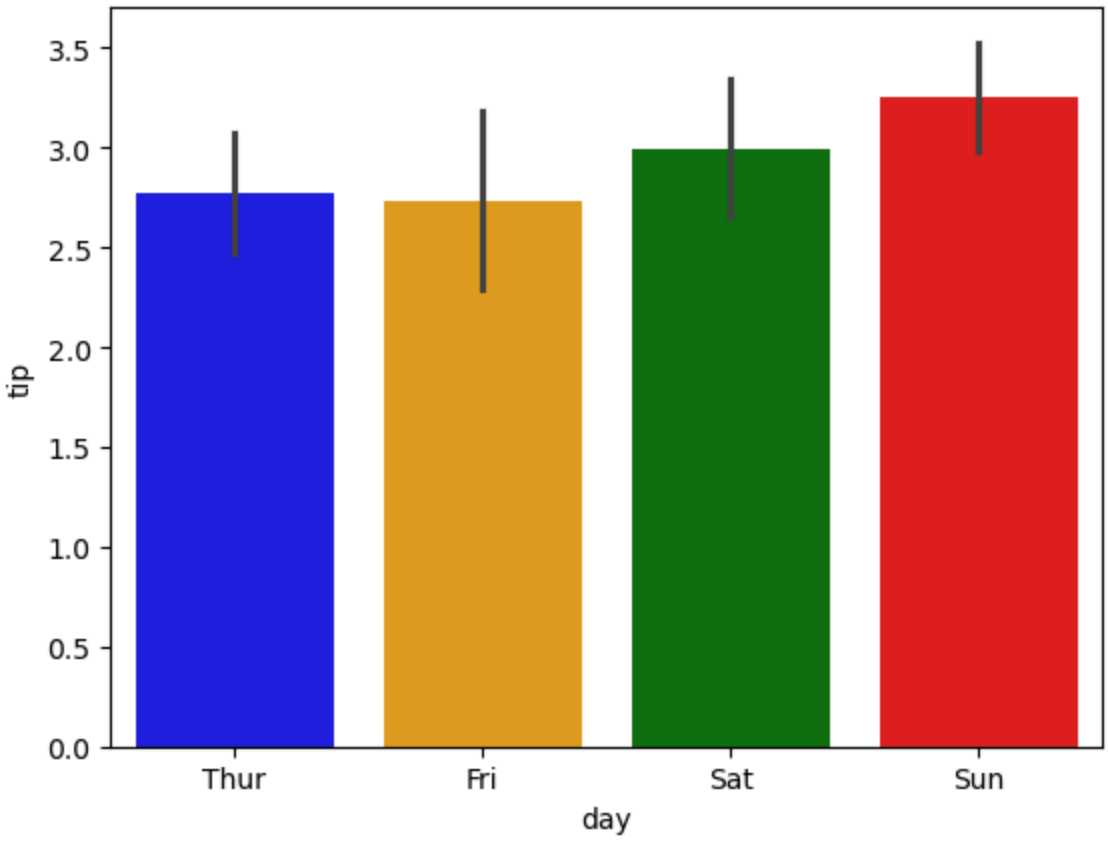
In [5]:
```
import week9hw as lib
lib.questionB("tip")
```



Question C Output

Here is the output for question C

In [1]:
```
import week9hw as lib
lib.questionC()
```

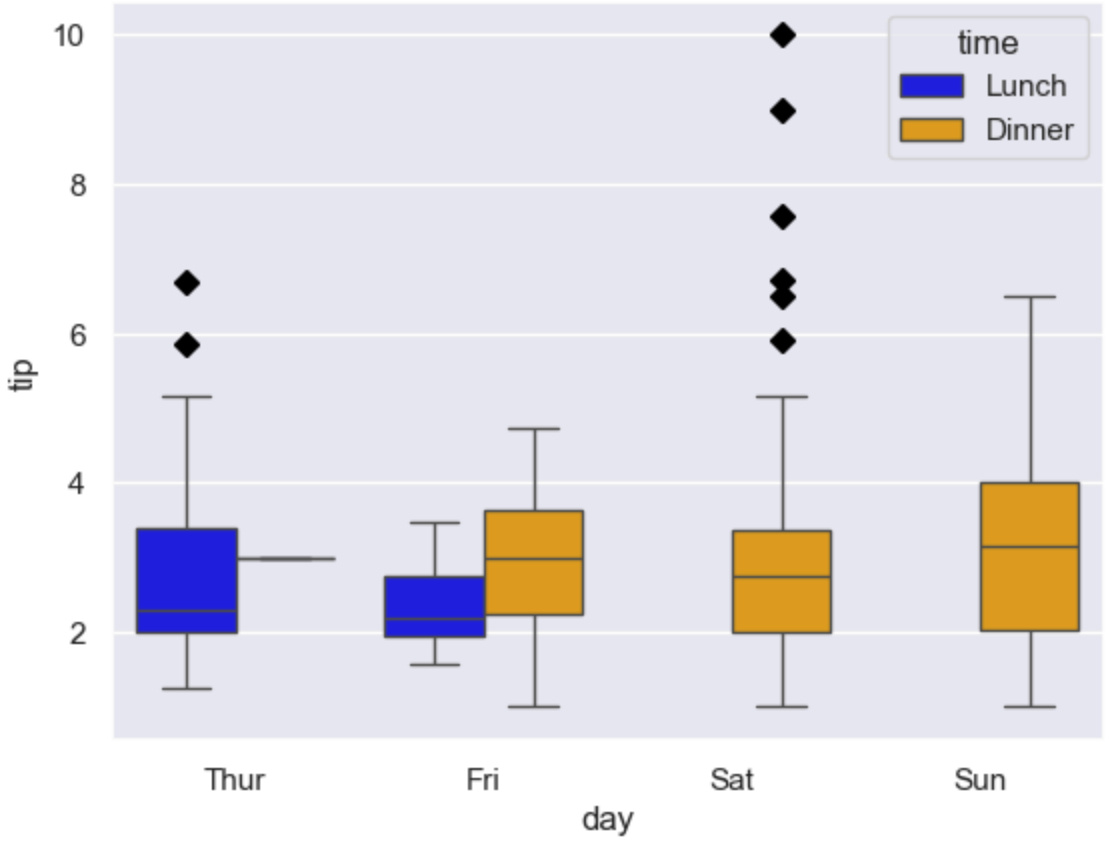


Question D Output

Here is the output for question D. We can derive the following information from this new visualization:

- The vast majority of tips for all days averaged between $2.00 and $4.00
- The restaurant is busy during lunch hours on Thursday, but does not appear to be open at dinner time.
- The reverse is true on Saturday and Sunday. It is busy during dinner time, but it does not appear to be open during lunch hours.
- Friday has a good mix of lunch and dinner customers, but dinner is busier
- The median tip value for dinner on Friday, Saturday and Sunday is a little over $3.00
- The most desirable shift appears to be Sunday evening. It appears to be the busiest with the maximum tip being $4.00, and the upper 25% of tips were somewhere around $3.50, which is higher than the maximum tip on some other days.

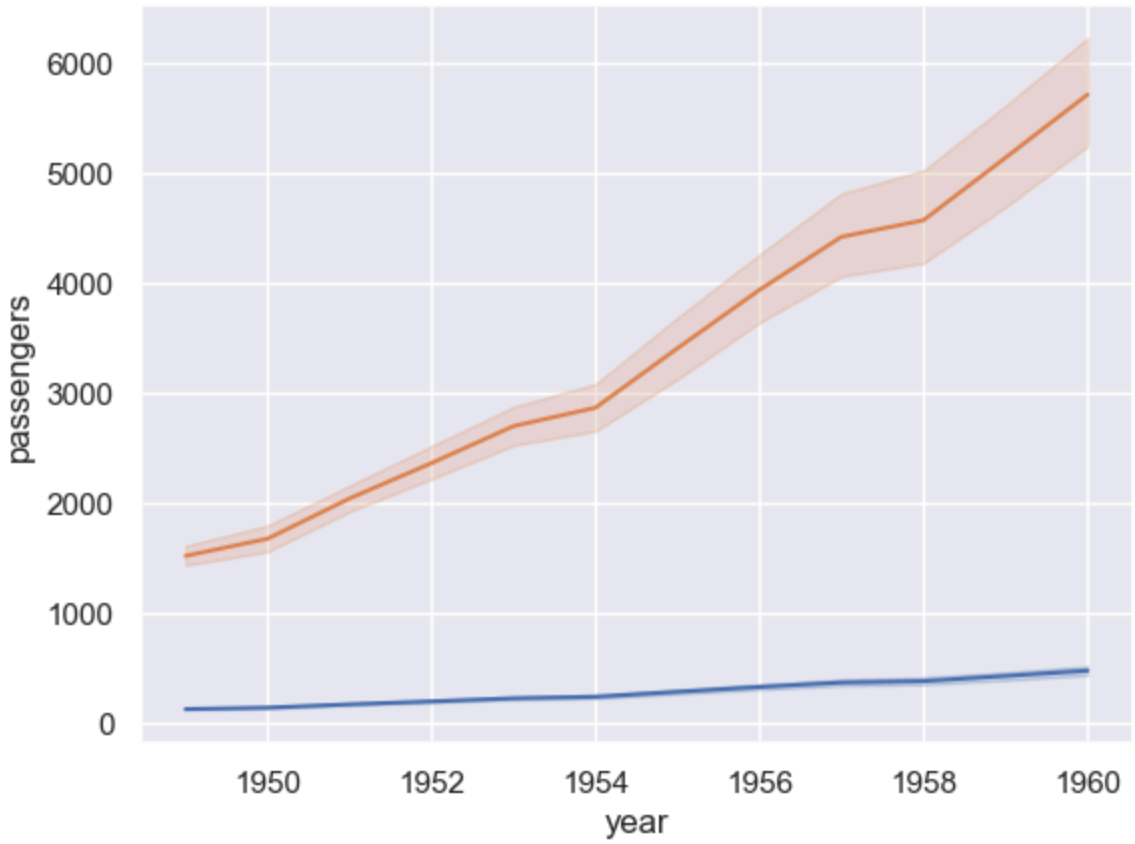
In [7]:
```
import week9hw as lib
lib.questionD()
```



Question E Output

Here is the output for question E

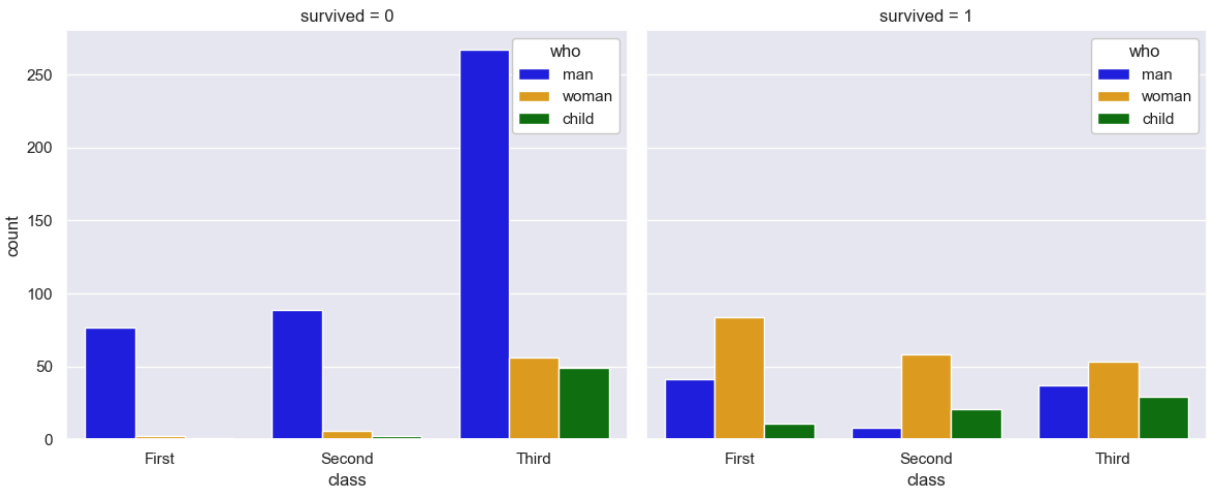
In [8]:
```
import week9hw as lib
lib.questionE()
```



Question F Output

Here is the output for question F

In [9]:
```
import week9hw as lib
lib.questionF()
```



In [ ]:
```

```